PayWallet

Hệ thống Ví điện tử & Thanh toán — Tài liệu chuẩn bị Demo phỏng vấn

Vị trí: **Backend Developer (Java/Spring)**, tham gia với tư cách Fresher. Tài liệu gồm: dự án làm gì, cách ánh xạ với JD, checklist chuẩn bị, **kịch bản demo 10 phút** (và bản rút gọn 3 phút), các điểm kỹ thuật, **bộ câu hỏi phỏng vấn kèm gợi ý trả lời** và những điểm yếu nên chủ động nói ra.

DEMO TRỰC TUYẾN	
Web demo	**https://sonddd.site**
Swagger API	https://sonddd.site/swagger-ui.html
Mã nguồn	https://github.com/sonddd17/payment-wallet-demo
Tài khoản	alice / alice123 · bob / bob123 (USER) · admin / admin123 (ADMIN)

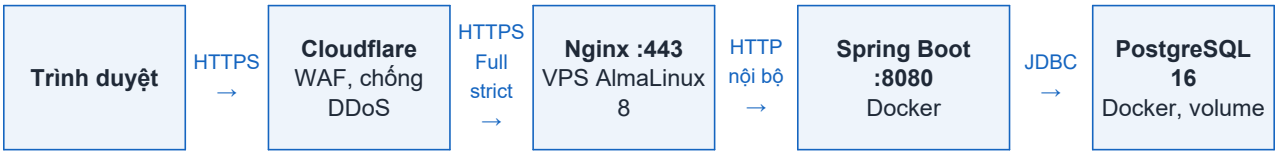
LƯU Ý	Dự án **mô phỏng** cho mục đích học tập/demo: **tiền ảo**, không kết nối cổng thanh toán thật, không xử lý thẻ hay tiền thật.

1. Trang web làm gì

Câu giới thiệu (học thuộc):

> *Em làm PayWallet, một hệ thống ví điện tử mô phỏng. Người dùng nạp tiền, chuyển tiền cho nhau, thanh toán cho cửa hàng và xem lịch sử; admin xem báo cáo. Backend viết bằng Spring Boot, dữ liệu lưu PostgreSQL, chạy bằng Docker trên VPS sau Nginx và Cloudflare, HTTPS suốt đường đi, tại sonddd.site.*

Đường đi của một request



Bên trong Spring Boot, request đi qua 4 tầng:

Tầng	Việc làm	File
Security filter	Đọc JWT trong header Authorization, xác định người gọi và vai trò	JwtAuthenticationFilter
Controller	Nhận HTTP, kiểm tra dữ liệu (@Valid), trả JSON chuẩn {success, message, data}	PaymentController…
Service	Nghiệp vụ: kiểm tra số dư, khóa ví, trừ/cộng tiền trong một transaction	PaymentServiceImpl
Repository	Spring Data JPA; Hibernate sinh SQL; lọc động bằng Specification	TransactionRepository

Mọi lỗi được GlobalExceptionHandler bắt và trả JSON lỗi với mã HTTP đúng nghĩa; lỗi 500 không lộ chi tiết nội bộ ra ngoài.

Dữ liệu (4 bảng)

Bảng	Nội dung	Quan hệ / điểm đáng nói
users	Tài khoản, mật khẩu băm BCrypt, vai trò USER/ADMIN	1-1 với wallets

Bảng	Nội dung	Quan hệ / điểm đáng nói
wallets	Số dư (BigDecimal), trạng thái, cột version	@Version: khóa lạc quan
merchants	Cửa hàng nhận thanh toán (COFFEE, SHOP)	Có số dư quyết toán riêng
transactions	Mọi giao dịch: loại, trạng thái, số tiền, mã tham chiếu	N-1 tới ví gửi, ví nhận, merchant; idempotency_key unique; index

Từng chức năng chạy thế nào

Chức năng	API	Bên trong
Đăng ký	POST /api/auth/register	Kiểm tra trùng, băm mật khẩu, tạo user + ví (số dư 0) trong một transaction
Đăng nhập	POST /api/auth/login	Spring Security so mật khẩu, trả JWT sống 24 giờ
Nạp tiền	POST /api/wallet/topup	Khóa ví, cộng tiền, ghi giao dịch TOPUP
Chuyển tiền	POST /api/payments/transfer	Chặn tự chuyển; khóa 2 ví theo thứ tự id; đủ số dư mới trừ/cộng; lỗi thì rollback toàn bộ
Thanh toán	POST /api/payments/pay	Như chuyển tiền, nhưng tiền vào tài khoản merchant
Lịch sử	GET /api/payments/history	Lọc động (loại, trạng thái, khoảng ngày) + phân trang
Báo cáo	GET /api/admin/report	GROUP BY type + COUNT + SUM; chỉ ADMIN, USER gọi nhận 403

2. Dự án đáp ứng JD như thế nào

Yêu cầu trong JD	Thể hiện trong dự án
OOP, cấu trúc dữ liệu & giải thuật	Phân tầng Controller–Service–Repository; interface + impl (trừu tượng, đa hình); entity đóng gói; cây exception kế thừa. Sắp xếp thứ tự khóa theo id để tránh deadlock; LinkedHashMap giữ thứ tự lỗi.
REST API	10 endpoint, response thống nhất, mã HTTP đúng nghĩa (200/201/400/401/403/404/409/422).
ORM (JPA, Hibernate)	4 entity, quan hệ 1-1 và N-1, @Version, index, pessimistic lock (SELECT … FOR UPDATE).
CSDL: truy vấn phức tạp, tối ưu	Lọc động bằng JPA Specification + phân trang; GROUP BY + SUM; index các cột tra cứu và sắp xếp; PostgreSQL thật.
Spring Boot, Core, MVC, Data JPA, Security	Constructor injection, @RestController, JpaRepository, SecurityFilterChain + JWT filter, @PreAuthorize.
Maven, Docker, Jenkins	pom.xml; Dockerfile + Docker Compose (đang chạy production); Jenkinsfile (test → package → build image).
JUnit, Mockito	4 unit test (Mockito); integration test MockMvc (đăng ký → nạp tiền → lịch sử có lọc); test đồng thời 10 luồng chuyển tiền cùng lúc.
Git	Repo public trên GitHub, commit rõ nghĩa, .gitignore bảo vệ bí mật.
SOAP (JD ghi “REST hoặc SOAP”)	Dùng REST. Hiểu SOAP dùng XML, WSDL/XSD; Spring hỗ trợ qua Spring-WS.

3. Checklist chuẩn bị

Hôm trước

- [ ] **Reset dữ liệu** trên web về trạng thái sạch (lệnh ở Phụ lục). Sau reset: alice 750.000, bob 500.000, 4 giao dịch.
- [ ] Mở thử https://sonddd.site và Swagger **trên đúng máy sẽ demo**.
- [ ] Mở IDE với project; đọc lại hàm **transfer()** trong PaymentServiceImpl — đoạn dễ bị hỏi nhất.
- [ ] Đọc lại mục 6 (câu hỏi) và mục 7 (điểm yếu). Tập nói câu giới thiệu thành tiếng 2–3 lần.

15 phút trước khi vào

- [ ] Mở sẵn 4 tab: **web** (đã đăng xuất) · **Swagger** · **GitHub repo** · **IDE** ở PaymentServiceImpl.
- [ ] Phóng to chữ trình duyệt và IDE (Ctrl +) để người xem đọc được; tắt thông báo.
- [ ] Thử đăng nhập alice một lần rồi đăng xuất, để chắc web phản hồi nhanh.

Phương án dự phòng

- **Mạng chặn hoặc lỗi trình duyệt (vd ERR_QUIC_PROTOCOL_ERROR):** đổi trình duyệt khác, hoặc dùng 4G điện thoại.
- **Không vào được web:** chạy bản local bằng jar dựng sẵn rồi mở http://localhost:8080 (dùng H2, có sẵn dữ liệu mẫu):

```
java -jar D:\JavaFS\payment-wallet-demo\target\payment-wallet-demo.jar
```

4. Kịch bản demo 10 phút

Nguyên tắc: **bấm → chỉ vào kết quả → nói một câu kỹ thuật**. Khung “Nhật ký phản hồi API” cuối trang hiện JSON server trả về — dùng nó để vừa thao tác vừa giải thích.

Bước 1 · Giới thiệu **0:00 – 0:30**

Làm: Mở https://sonddd.site.

Nói: “Em làm PayWallet, ví điện tử mô phỏng… (câu ở mục 1). Em chọn đề tài thanh toán vì nó buộc phải xử lý đúng giao dịch và truy cập đồng thời — hai vấn đề quan trọng của backend.”

Bước 2 · Kiến trúc **0:30 – 1:30**

Làm: Mở GitHub repo, cuộn tới sơ đồ “Kiến trúc triển khai” trong README.

Nói: “Request đi qua Cloudflare, tới Nginx trên VPS rồi vào Spring Boot chạy trong Docker, dữ liệu ở PostgreSQL. HTTPS suốt đường đi: Cloudflare ở chế độ Full strict, server dùng chứng chỉ Let's Encrypt tự gia hạn. Nginx chỉ nhận kết nối từ IP của Cloudflare, còn app chỉ mở cho localhost.”

Bước 3 · Đăng nhập bằng JWT **1:30 – 2:30**

Làm: Bấm nút “alice” → Đăng nhập. Chỉ vào accessToken trong khung nhật ký.

Nói: “Đăng nhập xong server trả JWT. Các request sau gửi kèm token trong header Authorization. Server không lưu session nên dễ chạy nhiều instance song song.”

Bước 4 · Nạp tiền và chuyển tiền (phần quan trọng nhất) **2:30 – 4:30**

Làm: Nạp 100.000 → số dư tăng. Chuyển 50.000 cho bob → giao dịch TRANSFER xuất hiện. Rồi mở code.

Nói: "Chuyển tiền nằm trong một @Transactional. Em lấy id hai ví trước, rồi khóa bằng pessimistic lock — SELECT FOR UPDATE — theo thứ tự id tăng dần để hai người chuyển chéo cho nhau không bị deadlock. Số dư chỉ được đọc SAU khi đã khóa, nên request đến sau sẽ chờ rồi thấy số dư mới nhất. Thiếu tiền hay lỗi giữa chừng thì toàn bộ rollback."

Mở code: PaymentServiceImpl.transfer() — đoạn findIdByUsername rồi findByIdForUpdate(min/max); WalletRepository.findByIdForUpdate (@Lock PESSIMISTIC_WRITE).

Bước 5 · Các trường hợp lỗi **4:30 – 5:30**

Làm: Chuyển 999999999 cho bob → lỗi 422 "Số dư không đủ". Nạp số âm → 400 kèm lỗi theo từng trường.

Nói: "Lỗi được xử lý tập trung ở GlobalExceptionHandler: mỗi loại lỗi trả mã HTTP đúng nghĩa. Lỗi hệ thống chỉ trả thông báo chung, chi tiết ghi vào log, không lộ SQL ra ngoài."

Bước 6 · Thanh toán và lịch sử **5:30 – 6:30**

Làm: Thanh toán 35.000 cho COFFEE → Tải lịch sử.

Nói: "Thanh toán trừ ví người dùng, cộng vào tài khoản merchant trong cùng một transaction. Mỗi giao dịch có mã tham chiếu duy nhất và hỗ trợ idempotency key để bấm hai lần không bị trừ hai lần."

Bước 7 · Phân quyền và báo cáo **6:30 – 7:30**

Làm: Đăng xuất → đăng nhập admin → Xem báo cáo.

Nói: "Báo cáo dùng GROUP BY và SUM theo loại giao dịch. Chỉ ADMIN gọi được; tài khoản USER gọi sẽ nhận 403. Em phân quyền ở cả cấu hình URL lẫn @PreAuthorize."

Bước 8 · Swagger: lọc động **7:30 – 8:30**

Làm: Mở Swagger → Authorize (dán token) → GET /api/payments/history với type=TRANSFER.

Nói: "Bộ lọc lịch sử ghép điều kiện động bằng JPA Specification: điều kiện nào có giá trị mới đưa vào câu SQL, kết hợp phân trang của Spring Data."

Bước 9 · Test và hai câu chuyện lỗi (phần ăn điểm) **8:30 – 9:30**

Làm: Mở ConcurrentTransferIntegrationTest và PaymentServiceImplTest trên GitHub; mở 2 commit "fix:" trong lịch sử commit.

Nói: "Lỗi thứ nhất: em viết test cho 10 luồng cùng chuyển tiền từ một ví và phát hiện 6/10 giao dịch hợp lệ bị từ chối. Nguyên nhân: ví đã được Hibernate nạp sẵn bằng truy vấn không khóa, nên câu SELECT FOR UPDATE trả về object cũ, không làm mới số dư; request sau tính trên số dư cũ và bị @Version chặn. Em sửa bằng cách lấy id trước, khóa rồi mới đọc; test giờ 10/10, và em đã thử lại trên PostgreSQL production. Lỗi thứ hai: trang lịch sử chạy được trên H2 nhưng lỗi 500 trên PostgreSQL vì truy vấn "tham số IS NULL"; em chuyển sang JPA Specification. Bài học: phải test đúng tình huống đồng thời, và môi trường test nên giống production."

Mở code: ConcurrentTransferIntegrationTest.java; TransactionSpecifications.java; 2 commit bắt đầu bằng "fix:".

Bước 10 · Kết	9:30 – 10:00

Làm: Quay lại trang web.

Nói: "Nếu có thêm thời gian em sẽ: chạy test trên PostgreSQL bằng Testcontainers, làm dashboard doanh thu realtime bằng SSE, xử lý N+1 query ở lịch sử, và cho Jenkins tự deploy. Em cảm ơn anh/chị đã lắng nghe."

Bản rút gọn 3 phút (khi bị giới hạn thời gian)

- **0:00** — Câu giới thiệu + một câu về kiến trúc HTTPS đầu–cuối.
- **0:30** — Đăng nhập alice, chuyển 50.000 cho bob, **mở hàm transfer()** giải thích transaction + khóa ví theo thứ tự.
- **2:00** — Kể câu chuyện test 10 luồng phát hiện lỗi khóa và cách sửa. Kết bằng hướng cải thiện.

5. Điểm kỹ thuật nổi bật

Chủ đề	Đã làm gì
Toàn vẹn giao dịch	@Transactional; khóa bi quan rồi mới đọc số dư; khóa theo thứ tự id chống deadlock; @Version làm lớp bảo vệ thứ hai; BigDecimal cho tiền; có test 10 luồng chứng minh.
Idempotency	Gửi lại cùng idempotency key thì trả giao dịch cũ; cột có unique constraint trong CSDL.
Bảo mật ứng dụng	JWT stateless, BCrypt, phân quyền URL + @PreAuthorize, bí mật đọc từ biến môi trường.
Xử lý lỗi	@RestControllerAdvice, mã HTTP đúng nghĩa, lỗi validation theo từng trường, xung đột đồng thời trả 409, không lộ lỗi nội bộ.
Truy vấn	JPA Specification cho lọc động, phân trang, GROUP BY + SUM, index trên các cột tra cứu/sắp xếp.
Kiểm thử	Unit test Mockito (4 ca nghiệp vụ chuyển tiền); integration test MockMvc toàn luồng; test đồng thời 10 luồng (đã phát hiện và sửa một lỗi khóa thật).
Hạ tầng	Docker Compose (app + PostgreSQL), Nginx, Let's Encrypt + Cloudflare Full strict, chỉ nhận IP Cloudflare, app bind localhost, swap + giới hạn heap cho VPS 1GB.

6. Câu hỏi hay gặp và gợi ý trả lời

Nghiệp vụ & giao dịch

Câu hỏi	Ý chính để trả lời
Hai người cùng rút tiền một ví cùng lúc?	Pessimistic lock (SELECT … FOR UPDATE): request sau phải đợi request trước commit rồi mới đọc số dư mới. Thêm @Version làm lớp bảo vệ thứ hai. Có test 10 luồng chuyển cùng lúc: tất cả thành công, số dư đúng.
Dùng @Lock rồi mà vẫn lỗi được không?	Có — em gặp rồi. Nếu entity đã được nạp vào persistence context bằng truy vấn không khóa, câu lệnh khóa trả về đúng object cũ mà không làm mới dữ liệu. Phải khóa trước rồi mới đọc (lấy id trước, hoặc entityManager.refresh(entity, PESSIMISTIC_WRITE)).
Pessimistic khác optimistic lock?	Pessimistic khóa trước khi sửa — hợp khi tranh chấp nhiều như tiền. Optimistic không khóa, lúc ghi kiểm tra version, ai chậm hơn bị lỗi và phải thử lại — hợp khi ít tranh chấp.

Câu hỏi	Ý chính để trả lời
Vì sao khóa theo thứ tự id?	A chuyển cho B và B chuyển cho A cùng lúc: nếu mỗi bên khóa ví mình trước sẽ chờ nhau mãi (deadlock). Khóa theo cùng một thứ tự thì không thể chờ vòng tròn.
Bấm “Chuyển” 2 lần có bị trừ 2 lần?	Client gửi idempotency key; key đã dùng thì trả lại giao dịch cũ. Nếu hai request cùng key đến đồng thời, unique constraint trong CSDL chặn request thứ hai (rollback, trả 409) nên tiền chỉ bị trừ một lần.
Vì sao dùng BigDecimal?	double sai số dấu phẩy động (0.1 + 0.2 ≠ 0.3), không chấp nhận được với tiền.
@Transactional hoạt động thế nào?	Spring tạo proxy bao quanh bean: mở transaction, commit khi thành công, rollback khi có RuntimeException. Gọi method trong cùng class (self-invocation) không đi qua proxy nên không có transaction.

Spring, CSDL & kiểm thử

Câu hỏi	Ý chính để trả lời
Dependency Injection là gì? Sao dùng constructor injection?	Spring tạo và tiêm phụ thuộc thay vì class tự new. Constructor injection: field final, không null, dễ viết test (truyền mock vào constructor) — đúng như cách viết PaymentServiceImplTest.
JWT khác session?	Session lưu trạng thái trên server. JWT tự chứa thông tin và chữ ký, server không lưu gì nên dễ scale. Nhược điểm: khó thu hồi trước hạn — giải pháp: token ngắn hạn + refresh token, hoặc danh sách đen.
N+1 query là gì?	Lấy N giao dịch rồi mỗi giao dịch lại truy vấn thêm ví/user. Sửa bằng JOIN FETCH hoặc @EntityGraph. Trang lịch sử hiện vẫn còn N+1 — em biết và đã ghi vào danh sách cải thiện.
Đánh index cột nào, vì sao?	username, email (unique, tra cứu khi đăng nhập); from_wallet_id, to_wallet_id (lọc lịch sử); created_at (lọc khoảng ngày và sắp xếp). Kiểm tra bằng EXPLAIN. Index tăng tốc đọc nhưng làm chậm ghi.
Vì sao chạy được trên H2 mà lỗi trên PostgreSQL?	Mỗi CSDL xử lý SQL hơi khác nhau; H2 dễ dãi với tham số null. Test nên chạy trên đúng loại CSDL production — dùng Testcontainers.
Unit test khác integration test?	Unit: một class, mock phụ thuộc, chạy nhanh. Integration: khởi động Spring context + CSDL, gọi API qua MockMvc, bắt lỗi khi các phần ghép với nhau.

Hạ tầng & thiết kế mở rộng

Câu hỏi	Ý chính để trả lời
Cloudflare Flexible khác Full (strict)?	Flexible: chỉ mã hóa khách → Cloudflare, đoạn Cloudflare → server là HTTP. Full (strict): mã hóa cả đoạn sau và kiểm tra chứng chỉ server hợp lệ. Em dùng Full (strict) + Let's Encrypt.
Sao phải chặn IP không phải Cloudflare?	Nếu không, ai biết IP server có thể gọi thẳng, bỏ qua Cloudflare (WAF, chống DDoS). Nginx chỉ nhận dải IP chính thức của Cloudflare và lấy IP thật của khách qua header CF-Connecting-IP.
Thanh toán xong muốn hiện doanh thu realtime: polling hay WebSocket?	Polling: đơn giản, trễ bằng chu kỳ hỏi, tốn request. SSE: server đẩy một chiều qua HTTP — vừa đủ cho dashboard. WebSocket: hai chiều, hợp chat/game, phức tạp hơn. Em chọn SSE.
Đẩy doanh thu lúc nào cho đúng?	Sau khi transaction commit: dùng @TransactionalEventListener(phase = AFTER_COMMIT), để không đẩy doanh thu của giao dịch bị rollback.
Chạy nhiều instance thì sao?	JWT stateless nên đăng nhập không vướng. Khóa ở CSDL nên vẫn đúng. Với SSE/WebSocket cần message broker (Redis Pub/Sub, RabbitMQ) để mọi instance nhận event.

Câu hỏi	Ý chính để trả lời
Có làm SOAP không?	Dự án dùng REST. SOAP dùng XML, hợp đồng WSDL/XSD; Spring hỗ trợ qua Spring-WS.

7. Điểm yếu nên chủ động nói (kèm hướng cải thiện)

Với fresher, biết rõ giới hạn của chính mình được đánh giá cao hơn giả vờ hoàn hảo. **Đừng nhận những gì chưa làm thật.**

Điểm còn yếu	Hướng cải thiện
Test chạy trên H2, không phải PostgreSQL	Testcontainers chạy PostgreSQL thật trong test
N+1 query ở trang lịch sử	@EntityGraph hoặc JOIN FETCH; hoặc projection DTO
Jenkinsfile mới viết sẵn, **chưa chạy trên Jenkins thật**; deploy vẫn làm tay	Dựng Jenkins/GitHub Actions: test → build → tự deploy lên VPS
Dockerfile multi-stage ở thư mục gốc **chưa build thử** (máy dev chưa có Docker); bản đang chạy production dùng deploy/Dockerfile	Build thử và đưa vào CI
Chưa giới hạn số lần đăng nhập sai; JWT không thu hồi được	Rate limiting (Bucket4j/Nginx), refresh token ngắn hạn
Một VPS, chưa sao lưu CSDL	pg_dump định kỳ ra nơi lưu trữ khác
Tài khoản demo có mật khẩu công khai	Chấp nhận được vì là demo; production thì không

8. Danh sách endpoint

Method	Đường dẫn	Chức năng	Quyền
POST	/api/auth/register	Đăng ký + tạo ví	Công khai
POST	/api/auth/login	Đăng nhập, nhận JWT	Công khai
GET	/api/wallet/me	Xem số dư	USER
POST	/api/wallet/topup	Nạp tiền	USER
POST	/api/payments/transfer	Chuyển tiền	USER
POST	/api/payments/pay	Thanh toán merchant	USER
GET	/api/payments/history	Lịch sử (type, status, fromDate, toDate, page, size)	USER
GET	/api/merchants	Danh sách merchant	Công khai
GET	/api/admin/transactions	Toàn bộ giao dịch	ADMIN
GET	/api/admin/report	Báo cáo tổng hợp	ADMIN

Phụ lục. Lệnh hữu ích

Reset dữ liệu web về trạng thái sạch (xóa toàn bộ dữ liệu trên sonddd.site, kể cả tài khoản người khác đăng ký; đợi khoảng 1 phút rồi mở lại trang):

```
ssh root@103.116.39.23 "cd /opt/paywallet/deploy && docker compose -f
docker-compose.prod.yml down -v && docker compose -f docker-compose.prod.yml up -d"
```

Chạy bản local (không cần mạng tới server):

```
java -jar D:\JavaFS\payment-wallet-demo\target\payment-wallet-demo.jar
```

Xem log ứng dụng trên server:

```
ssh root@103.116.39.23 "docker logs --tail 100 paywallet-app"
```

Chúc bạn demo thành công!